

a. Události, které lze obsloužit skriptem

tabulka 1 Události

OnLoad	Událost je vyvolána po otevření dokumentu v okně prohlížeče nebo v rámu v rámci jednoho FRAMESET . Atribut může být použit u elementů BODY a FRAMESET .
OnUnload	Událost je vyvolána po odstranění dokumentu z okna nebo rámu. Atribut může být použit s elementy BODY a FRAMESET .
OnClick	Událost je vyvolána po kliknutí myši na element. Atribut může být použit u většiny elementů.
OnDblClick	Událost je vyvolána po dvojitém kliknutí myši na element. Atribut může být použit u většiny elementů.
OnMouseDown	Událost je vyvolána po stisknutí tlačítka myši nad elementem. Atribut může být použit u většiny elementů.
OnMouseUp	Událost je vyvolána po uvolnění tlačítka myši nad elementem. Atribut může být použit u většiny elementů.
OnMouseOver	Událost je vyvolána při přesunutí myši nad elementem. Atribut může být použit u většiny elementů.
OnMouseMove	Událost je vyvolána při pohybu myši nad elementem. Atribut může být použit u většiny elementů.
OnMouseOut	Událost je vyvolána po odsunutí myši z elementem. Atribut může být použit u většiny elementů.
OnFocus	Událost je vyvolána v okamžiku, kdy je element aktivován myši nebo pomocí tabulátoru. Atribut je možné použít u elementů LABEL , INPUT , SELECT , TEXTAREA a BUTTON .
OnBlur	Událost je vyvolána v okamžiku, kdy element přestává být aktivní. Atribut je možné použít u elementů LABEL , INPUT , SELECT , TEXTAREA a BUTTON .
OnKeyPress	Událost je vyvolána po stisku a následném uvolnění tlačítka na klávesnici. Atribut může být použit u většiny elementů.
OnKeyDown	Událost je vyvolána po stisku tlačítka na klávesnici. Atribut může být použit u většiny elementů.
OnKeyUp	Událost je vyvolána po uvolnění tlačítka na klávesnici. Atribut může být použit u většiny elementů.
OnSubmit	Událost je vyvolána při odesílání formuláře. Atribut může být použit pouze u elementu FORM .
OnReset	Událost je vyvolána po vynulování formuláře. Atribut může být použit pouze u elementu FORM .
OnSelect	Událost je vyvolána po označení textu ve výstupním poli. Atribut může být použit pouze u elementů INPUT a TEXTAREA .
OnChange	Událost je vyvolána pokud se změnila hodnota vstupního pole formuláře. Atribut může být použit pouze u elementů INPUT , SELECT a TEXTAREA .

1. Příklad události onLoad

Po otevření této stránky se aktivuje událost onLoad, která zobrazí okno s nápisem vítejte.

<HTML>

<HEAD>

<TITLE>onLoad</TITLE>

```
</HEAD>
<BODY onLoad="window.alert('vítejte');">
</BODY>
</HTML>
```

b. Spouštění funkcí a příkazů v JavaScriptu

1. Zadání příkazu přímo do odkazu HTML

Příklad spust'

2. Zavolání JavaScriptové funkce z odkazu HTML

spust'

3. Pomocí událostí - například onclick

a. Klepnutím na obrázek s názvem obrazek.gif se spustí funkce **spocitej()**

Příklad

b. Klepnutím na odkaz se spustí funkce **otevri_okno()**

Příklad okno

c. Datum a čas

Pro práci s datem a časem je potřeba nejprve vytvořit nový objekt typu Date. Pak je možné z tohoto nového objektu získávat jednotlivé informace o času.

cas=new Date();

Pak již lze pomocí metod a vlastností přistupovat k novému objektu **cas**.

i. Metody objektu Date

getDate() Vrací číslo dne uloženého v instanci objektu Date v tomto měsíci.

getDay() Vrací pořadové číslo dne v týdnu uloženém v instanci objektu Date. Číslo je v rozmezí od 0 (neděle) do 6 (sobota)

getFullYear vrací položku rok v čtyřciferném tvaru

getHours() vrací číslo v rozmezí 0 (půlnoc) až 23, které vyjadřuje hodinu právě uloženou v instanci objektu Date

getMilliseconds vrací položku milisekundy

getMinutes() vrací číslo v rozmezí 0 až 59, které vyjadřuje minutu právě uloženou v instanci objektu Date

getMonth() vrací číslo v rozmezí 0 (leden) až 11 (prosinec), které vyjadřuje měsíc právě uložený v instanci objektu Date

getSeconds() vrací číslo v rozmezí 0 až 59, které vyjadřuje sekundu právě uloženou v instanci objektu Date

getTime() vrací číslo, které vyjadřuje počet milisekund, které uplynuly od 1. 1. 1970 do data uloženého právě v instanci objektu Date. Může sloužit při porovnávání dvou dat

getTimezoneOffset() vrací rozdíl (v minutách) mezi datem uloženým v instanci objektu Date a GMT (časem, který je právě teď v Grennwiche)

getFullYear() vrací rok ve tvaru RR (97)

setDate(x) nastavení dne v měsíci pro instanci objektu Date, kde x je hodnota v rozmezí 1 až 31

setDay nastaví den v měsíci

SetFullYear vrací položku rok v čtyřciferném tvaru

setHours(x) nastavení hodiny pro instanci objektu Date, kde x je hodnota v rozmezí 0 (půlnoc) až 23

setMilliseconds nastaví položku milisekundy

setMinutes(x) nastavení minut pro instanci objektu Date, kde x je hodnota v rozmezí 0 až 59

setMonth nastaví položku měsíc

setSeconds nastaví položku sekundy

setTme(x) nastavení doby, která uplynula od 1.1.1970 pro instanci objektu Date, kde x je číslo vyjadřující počet milisekund

setYear(x) nastavení roku pro aktuální instanci objektu Date, kde x je hodnota vyjadřující rok aktuálního letopočtu

toGMTString() převádí čas uložený v instanci objektu Date na čas používající formát GMT

toString() vrací řetězec reprezentující objekt data

toLocaleString() převádí čas uložený v instanci objektu Date na lokální čas. Je tedy metodou přesně opačnou k metodě předchozí

UTC() převádí datum a čas (v lokálním formátu) na počet milisekund od 1.1.1970

parse() převádí datum a čas (ve formátu GMT) na počet milisekund od 1.1.1970

Příklad – výpis čísla dne v měsíci

```
dnes=new Date();  
document.write(dnes+"<BR>");  
document.write(dnes.getDay());
```

ii. Načasování v JavaScriptu

K načasování slouží příkaz **window.setTimeout(„co se stane“,čas v ms)**

1. Příklad načasování v JavaScriptu

```
<HTML>  
<HEAD>  
<TITLE>Načasování v JavaScriptu</TITLE>  
<SCRIPT LANGUAGE="JavaScript">  
<!--  
function obnov()  
{  
    window.setTimeout(window.location="soubor.htm",3000);  
}  
// -->  
</SCRIPT>  
</HEAD>  
<BODY onLoad="obnov();">
```

Každé 3 sekundy se tato stránka znovu nahraje
</BODY>
</HTML>

Poznámka

window.location – tato funkce objektu window nahraje do okna stránku.

window.clearTimeout(ukazatel na window.setTimeout) – zruší window.setTimeout

2. Příklad hodin zobrazených ve formuláři

```
<HTML>
<HEAD>
<TITLE>Hodinky</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
```

```
function zapni_hodinky()
{
    dnes=new Date();
    with (dnes) {
        hodina=getHours(); minuta=getMinutes(); sekunda=getSeconds(); }
    cas=hodina;
    cas+=((minuta<10) ? ":0" : ":")+minuta;
    cas+=((sekunda<10) ? ":0" : ":")+sekunda;
    document.F1.T1.value=cas;
    casovac=window.setTimeout("zapni_hodinky()",1000);
}
// -->
</SCRIPT>
</HEAD>
<BODY onload="zapni_hodinky();">
<FORM name="F1">
<INPUT TYPE="text" name="T1" width=8>
</FORM>
</BODY>
</HTML>
```

Výsledkem jsou pak hodiny

19:07:48

d. JavaScriptový objektový model

JavaScript je založen na jednoduchém objektově orientovaném vzoru. Objekt je konstrukce s vlastnostmi, které jsou JavaScriptovými proměnnými. Vlastnosti mohou být také i jiné objekty. Funkce, které jsou asociovány s objekty, jsou známy jako objektové metody.

Objekty a vlastnosti

S JavaScriptovým objektem jsou neoddelitelně spojeny vlastnosti. Přístup na tyto vlastnosti je přes jednoduchou tečkovou notaci :

objectName.propertyName

Jméno objektu i vlastnosti jsou "case sensitive". Vlastnost se definuje přiřazením hodnoty. Například. předpokládejme, že už máme vytvořeny objekt `mojeauto` a můžeme mu přidat vlastnosti výrobce, model, rok takto:

```
mojeauto.vyrobce = "Ford"
mojeauto.model = "Mustang"
mojeauto.rok = 69;
```

Pole je uspořádaná množina hodnot spojených s jedním jménem proměnné. Vlastnosti a pole jsou si v JavaScriptu velmi blízké. Mají rozdílný interface pro stejnou datovou strukturu. Takže například můžeme získat vlastnosti objektu **mojeauto** takto :

```
mojeauto ["vyrobce"] = "Ford"
mojeauto ["model"] = "Mustang"
mojeauto ["rok"] = 67;
```

Naprosto ekvivalentně i tímto způsobem

```
mojeauto [0] = "Ford"
mojeauto [1] = "Mustang"
mojeauto [2] = 67;
```

Další možnost je použít cyklus for pro výpis všech vlastností

```
function ukaz_vlastnosti(obj, obj_name)
{ var result = ""
for (var i in obj)
result += obj_name + "." + i + " = " + obj[i] + "\n"
return result;
}
```

Volání funkce `ukaz_vlastnosti (moje auto, " moje auto ")` vrací následující:

```
mojeauto.vyrobce = Ford
mojeauto = Mustang
mojeauto.rok = 67
```

Metody

Metoda je funkce asociována s objektem. Metodu definujeme úplně stejně, jako definujeme standardní funkci. Pak použijeme následující syntaxi na spojení funkce s existujícím objektem:

object.methodname = function_name

kde **object** je existující objekt, **methodname** je jméno, které jsme přiřadili metodě a **function_name** je jméno funkce.

Volání metody v kontextu s objektem

```
object.methodname(params);
```

Použití "this" pro objektové reference

JavaScript má speciální klíčové slovo `this`, které můžeme použít k odkazu na současný objekt. Tak například předpokládejme funkci nazvanou **validate**, která zvaliduje hodnotu vlastnosti objektu, při zadání objektu a nejnižší a nejvyšší hodnoty:

```
function validate(obj, lowval, hival)
{
if ((obj.value < lowval) || (obj.value > hival))
alert("Invalid Value!")
}
```

Pak můžeme zavolat **validate** v každém prvku formuláře při použití `onChange` event handleru a pomocí `this` předat prvek formuláře. Obecně se v metodě `this` odkazuje na volající objekt.

Vytvoření nového objektu

Novou instanci objektu vytvoříme pomocí **new**. Chceme-li definovat objekt, vytvoříme funkci pro tento objektový typ, která bude specifikovat jeho jméno, vlastnosti a metody. Například

předpokládejme, že chceme vytvořit objekt pro auta. Chceme aby se tento typ objektu jmenoval **auto** a jeho vlastnosti byly výrobce, model, rok, a barva.

Vytvoření funkce

```
function auto(vyrobce, model, rok)
{
  this.vyrobce = vyrobce;
  this.model = model;
  this.rok = rok;
}
```

Použití `this` nám přiřadí hodnoty k objektovým vlastnostem podle hodnot předaných funkci.

Vytvoření objektu **mojeauto**:

```
mojeauto = new auto("Eagle", "Talon TSi", 1993);
```

Tento příkaz vytvoří **mojeauto** a přiřadí mu specifické hodnoty pro jeho vlastnosti. Pak hodnotou **mojeauto.vyrobce** je řetězec "Eagle", **mojeauto.rok** celé číslo 1993.

Můžeme vytvořit jakékoliv množství objektu typu **auto** dalšími voláními `new`.

```
kenscar = new auto("Nissan", "300ZX", 1992)
```

Objekt může mít vlastnost, která je sama o sobě jiný objekt.

Například předpokládejme objekt nazvaný **osoba** podle následujícího:

```
function osoba(jmeno, vek, pohlavi) {
  this.jmeno = jmeno;
  this.vek = vek;
  this.pohlavi = pohlavi;
}
```

Inicializace nových osob (objekty):

```
novak = new osoba("Novak Jan", 33, "M")
```

```
Kučera = new osoba("Kučera Petr", 39, "M")
```

Pak můžeme přepsat definici objektu **auto** a zahrnout do něj vlastnost majitele, která využije objekt **osoba**:

```
function auto(vyrobce, model, rok, vlastnik)
{
  this.vyrobce = vyrobce;
  this.model = model;
  this.rok = rok;
  this.vlastnik = vlastnik;
}
```

Pro inicializaci nových objektu použijte následující:

```
auto1 = new auto("Eagle", "Talon TSi", 1993, rand);
```

```
auto2 = new auto("Nissan", "300ZX", 1992, ken);
```

Nalezení jména majitele `auto2`

```
car2.Vlastnik..name
```

i. Přidávání vlastnosti už k nadefinovanému objektu

```
auto1.color = "black"
```

Přidá se vlastnost barvy ke `car1` a přiřadí ji hodnotu "black". Nicméně toto nemění jakýmkoliv způsobem jiné objekty. Pokud chcete přidat novou vlastnost všem objektům daného typu, pak musíte přidat tuto vlastnost do definice tohoto objektového typu.

Definování metod

Můžeme definovat metody pro objekt přidáním definice metody do definice daného objektového typu. Například předpokládejme, že máme množinu obrázku gif a chceme definovat metodu, která zobrazí informace pro auta vedle odpovídajícího obrázku. Pak můžeme nadefinovat funkci:

```
function displayCar() {
```

```

var result = "A Beautiful " + this.rok
            + " " + this.vyrobce + " " + this.model;
pretty_print(result)
}

```

Pretty_print je už jednou definována funkce na zobrazení řetězce. Připomeňme, že použití `this` odkazuje na objekt ke kterému metoda patří. Tuto funkci můžeme udělat metodou **auto** přidáním příkazu k definici objektu: `this.displayCar = displayCar.`

Plná definice auto:

```

function auto(vyrobce, model, year, vlastnik)
{
  this.vyrobce = vyrobce;
  this.model = model;
  this.rok = rok;
  this.vlastnik = vlastnik;
  this.displayCar = displayCar;
}

```

Pak můžeme volat tuto novou metodu:

```
auto1.displayCar()
```

```
auto2.displayCar()
```

Výsledkem je: A Beautiful 1993 Eagle Talon TSi

A Beautiful 1992 Nissan 300ZX

Vnitřní objekty a funkce

Funkce `parseInt` a `parseFloat`

Tyto dvě vnitřní funkce vrací numerickou hodnotu, pokud je jim dán řetězec jako parametr.

`parseFloat` parsuje svůj argument, řetězec, a pokouší se vrátit reálné číslo, jestliže narazí na znak jiný než `+` či `-`, číslice 0-9, a desetinnou tečku či exponent, pak vrátí hodnotu až do tohoto bodu a ignoruje vše následující. Jestliže první znak nemůže být zkonvertován na číslo, vrátí **NaN**.

`parseInt` parsuje svůj první argument a pokouší se vrátit celé číslo daného základu. Například základ 10 indikuje konverzi do desítkové soustavy a obdobně pro hexadecimální, oktalovou. Pro základ nad 10 jsou použita písmena abecedy počínaje A.

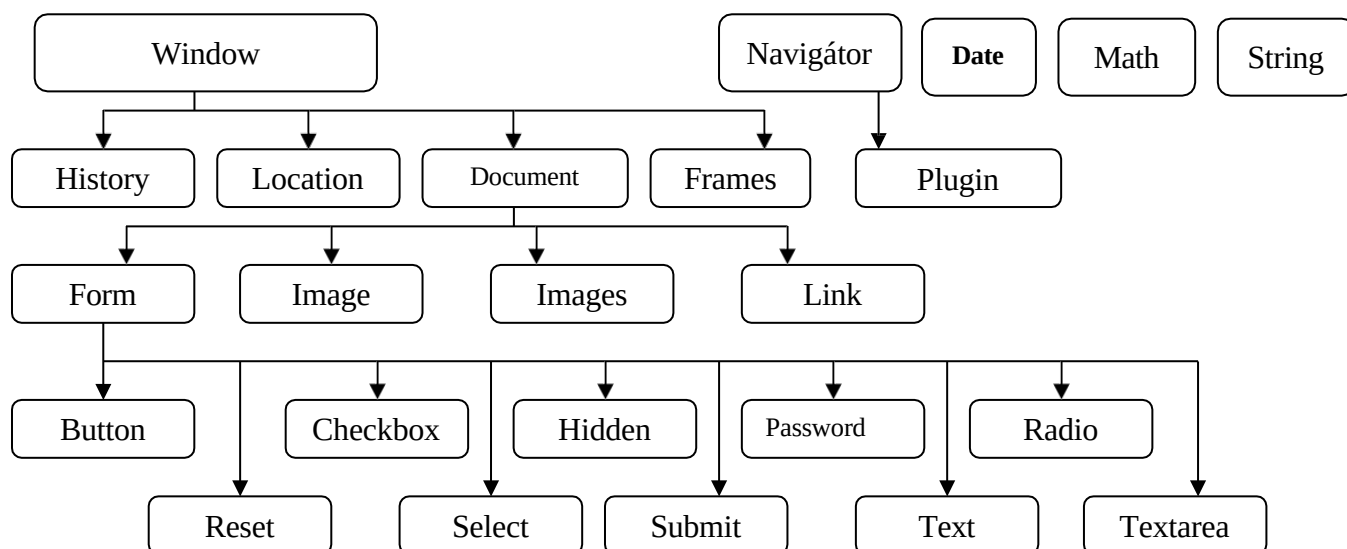
Jestliže narazí na znak jiný který je přípustný pro daný základ vrátí hodnotu až do tohoto bodu a ignoruje vše následující. Jestliže první znak nemůže být zkonvertován na číslo daného základu, vrátí **NaN**.

ii. Hierarchie objektů JavaScript

Použití

objekt.metoda

objekt.vlastnost



e. Metody objektu Array

concat	spojí dvě anebo více polí dohromady
join	převéde pole na řetězec prvků oddělených určeným oddělovačem
pop	odstraní prvek z pole
push	přidá jeden anebo více prvků na konec pole
reverse	obrátil pořadí prvků v poli
shift	odstraní první prvek z pole
slice	vrátí specifikovanou část pole
sort	seřadí prvky pole
splice	odstraní určený počet prvků z pole a současně přidá nové prvky
tostring	přidá řetězec obsahující všechny prvky pole oddělené čárkou
unshift	přidá jeden/více prvků na začátek pole

Vlastnosti objektu Array

length velikost pole

Metody objektu Document

Close uzavře výstup do dokumentu

Open otevře vstup do dokumentu

Write zapíše data do dokumentu

writeln zapíše data do dokumentu a odřádkuje

Vlastnosti objektu document

AlinkColor barva aktivovaných odkazů

all pole všech objektů tvořících dokument (<A NAME>)

bgColor barva pozadí dokumentu

charset aktuální znaková sada (kódování) dokumentu

cookie řetězec obsluhující cookies příslušející danému dokumentu

fgColor barva popředí dokumentu

forms[] pole formulářů tvořících dokument (<FORM>)

images[] pole obrázků tvořících dokument

lastModified datum a čas poslední změny dokumentu

linkColor barva nenavštívených odkazů

links[] pole odkazů tvořících dokument (<A HREF>)

location aktuální URL dokumentu

referer	URL dokumentu, z kterého jste se dostali na aktuální dokument
styleSheets[]	pole kaskádových stylů tvořících dokument
title	titulek dokumentu
URL	URL aktuálního dokumentu
URLUnencoded	dekódované URL aktuálního dokumentu
vlinkColor	barva nenavštívených odkazů

Vlastnosti objektu event

altKey	logická hodnota, která určuje, zda je stisknuta klávesa ALT
button	údaj o tom, které tlačítko myši je stlačeno
clientX	x-ová souřadnice kurzoru myši vzhledem k pracovní ploše prohlížeče
clientY	y-ová souřadnice kurzoru myši vzhledem k pracovní ploše prohlížeče
ctrlKey	logická hodnota, která určuje, zda je stisknuta klávesa Ctrl
keyCode	klávesová kód Unicode stisknuté klávesy
returnValue	návratová hodnota události
screenX	x-ová souřadnice kurzoru myši vzhledem k celkové ploše obrazovky
screenY	y-ová souřadnice kurzoru myši vzhledem k celkové ploše obrazovky
shiftKey	logická hodnota, která určuje, zda je stisknuta klávesa Shift
type	typ události, která právě nastala

Metody objektu location

reload	znovu načte aktuální dokument
replace	nahradí aktuálně zobrazený dokument

jiným Vlastnosti objektu location

hash	řetězec začínající znakem # , který specifikuje název kotvy v URL
host	řetězec obsahující název hostitele a portu URL
hostname	řetězec obsahující název hostitele URL
href	řetězec obsahující kompletní URL adresu
pathname	řetězec obsahující část URL, která specifikuje cestu
port	řetězec obsahující část URL, která představuje komunikační port
protocol	řetězec obsahující protokolovou část URL
search	řetězec začínající znakem ?, který specifikuje dotazovací část URL

i. Vlastnost window.status

Status je ke čtení i psaní řetězcová hodnota, která specifikuje přechodovou zprávu, která se má objevit na stavové řádce. Zpravidla se objevuje pouze na omezenou dobu – dokud není přepsána jinou zprávou nebo dokud uživatel nepřesune myš na nějakou jinou oblast okna. Když je zpráva, specifikovaná pomocí **status**, vymazána, stavový řádek se vrací do svého implicitního čistého stavu nebo k implicitní zprávě, specifikované vlastností **defaultStatus**. Ačkoli pouze okna nejvyšší úrovně mají stavový řádek, může být nastavena také vlastnost **status** rámečků. Tím se zobrazí specifikovaná zpráva ve stavovém řádku okna nejvyšší úrovně. Přechodné zprávy nastavené rámečky budou viditelné bez ohledu na to, který rámeček má v současné době zaměření nebo ve kterém je myš. To se liší od chování **defaultStatus**.

Použití windows.status

Obecně je použití vlastnosti **status** užitečné pouze z událostních ovladačů a ve fragmentech kódu, odloženého pomocí metody `Window.SetTimeout( )`. Jestliže nastavíte **status** přímo ze skriptu, zpráva nebude pro uživatele viditelná, nebude zobrazena okamžitě a když bude zobrazena, bude pravděpodobně ihned přepsána nějakou zprávou prohlížeče jako například „Document: done“.

Jestliže chcete nastavit vlastnost **status** v událostním ovladači **onMouseOver()** hypertextového spojení, musíte z tohoto událostního ovladače vrátit **true**. Je to proto, že implicitní akce, jde-li přes spojení, je zobrazení URL tohoto spojení, čímž se přepíše jakákoli stavová zpráva nastavená událostním ovladačem. Vrácením **true** z událostního ovladače rušíte tuto implicitní akci a zanecháváte zobrazenou svou vlastní zprávu **status** (dokud se myš nepřesune pryč ze spojení).

Příklad

```
<HTML><HEAD>
</HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!--
    window.defaultStatus="Vítejte na naší škole";
// -->
</SCRIPT>
<BODY>
<B>Zajímavé linky:</B><BR>
<A HREF="http://www.cpress.cz"
onMouseOver="window.status='Knižní vydavatelství CPRESS';return
true;">CPRESS</A><BR>
<A HREF="http://def.vosji.cz/smrcka/"
onMouseOver="window.status='Domovská stránka autora '; return true;">Smrčka</A>
</BODY></HTML>
```

f. Cookie

Vlastnost Document.cookie

Cookie je řetězcová vlastnost, umožňující číst, vytvářet, modifikovat a mazat cookie použité v aktuálním dokumentu. Cookie je malé množství pojmenovaných dat uložených webovým prohlížečem. To poskytuje webovému prohlížeči jakousi paměť, což umožňuje použít datový vstup na jedné stránce v jiné stránce nebo změnit uživatelské nastavení všech prohlížečových seancí..

Je-li to potřebné, jsou data cookie automaticky přenášena mezi webovým prohlížečem a webovým serverem., takže CGI-skripty na serveru mohou číst a zapisovat hodnoty cookie. Také kód JavaScriptu na straně klienta může pomocí této vlastnosti číst a zapisovat cookie. Každá cookie se skládá z těchto částí

doména serveru např. vosji.cz

údaj o tom, zda se jedná o **cookie využívající protokol https** nebo ne
adresářová struktura, která má přístup ke cookie

datum platnosti

název proměnné

hodnota proměnné

i. Čtení hodnot cookie

Pokud se použije vlastnost cookie ve výrazu JavaScriptu, je vrácenou hodnotou řetězec, obsahující všechny cookie, použité v aktuálním dokumentu. Řetězec je seznam párů **name=value**, oddělených středníky, kde **name** je název cookie a **value** je její řetězcová hodnota.

Cookie načteme příkazem

obsah=document.cookie;

Pak musíme jednotlivé dvojice **proměnná=hodnota** oddělit od středníků do pole pomocí příkazu `split`:

```
upravene=obsah.split(';')
```

Dále oddělíme jméno od hodnoty

```
velikost=upravene.length
```

Hodnota cookie nesmí obsahovat žádné středníky, čárky ani neviditelné znaky (tabulátory, mezery, nové řádky). Protože jde o často používané znaky, používá se v JavaScriptu běžně funkce **escape()**, která hodnoty cookie před jejich uložením zakóduje a funkce **unescape()**, která hodnoty dekóduje po jejich opětovném získání.

Document.cookie neposkytuje žádný způsob, jak získat položky domény, cesty, data platnosti nebo bezpečnostní, připojené ke cookie.

Ukládání cookie

K připojení dočasné hodnoty cookie k aktuálnímu dokumentu se nastaví **document.cookie** na řetězec ve tvaru **name=value**. Při dalším čtení `document.cookie` bude pár **name** a **value** začleněn v seznamu cookie tohoto dokumentu.

Cookie zapsané výše uvedeným způsobem, budou trvat po dobu aktuální prohlížečové seance, ale zmizí, když uživatel ukončí prohlížeč. Abyste vytvořili cookie, které mohou přetrvávat více prohlížečových seancí, musíte vložit datum ukončení platnosti (expiration date). To můžeme provést nastavením **document.cookie** na řetězec ve tvaru:

```
name=value; expires=date
```

Date musí být ve formátu zapsaném prostřednictvím `Date.toGMTString( )`.

Podobně můžeme nastavit cestu, doménu a bezpečnostní položku pro cookie připojením řetězců v následujícím tvaru k hodnotě cookie před zapsáním hodnoty do vlastnosti

document.cookie.

```
; path=path
```

```
; domain=domain
```

Změny hodnoty cookie dosáhnete opětovným nastavením hodnoty s použitím téhož názvu a nové hodnoty.

Vymazání cookie dosáhnete použitím téhož názvu, cesty a domény, libovolné hodnoty a prošlého data platnosti. Prohlížeč není povinen okamžitě smazat prošlá cookie.

ii. Příklad funkce pro přečtení hodnoty cookie

```
function prectiCookie(nazev)
```

```
{
```

```
    obsah=document.cookie; upravene=obsah.split(';');
```

```
    velikost=upravene.length; kusy=""; vrat="";
```

```
    for (x=0;(x<velikost && vrat=="");x++)
```

```
    {
```

```
        kusy=upravene[x].split('=');
```

```
        if (kusy[0].substring(0,1)==' ')
```

```
        kusy[0]=kusy[0].substring(1,kusy[0].length);
```

```
        if (kusy[0]==nazev) vrat=kusy[1];
```

```
    }
```

```
    if (vrat) return unescape(vrat);
```

```
    else return false;
```

```
}
```

Poznámka - funkce `split` rozdělí řetězec podle oddělovače do pole.

Příklad funkce pro uložení cookie

```
function zapisCookie(nazev,hodnota,vyprsi)
```

```
{
```

```

datum=new Date();
if(vyprsi) {
datum.setTime(datum.getTime()+vyprsi);
document.cookie=nazev+"="+escape(hodnota)
+"; expires="+datum.toGMTString(); }
else document.cookie=nazev+"="+escape(hodnota);
}

```

Použití

Cookie jsou zamýšleny pro nepřiliš časté ukládání malých množství dat. Nejsou pro univerzální komunikační nebo programovací mechanismus. Po webových prohlížečích není vyžadováno podržet v paměti více než 20 cookie na webový server, ani více než 4 kilobajty délky cookie párů **name/value**.

g. Metoda Function.toString()

Konvertuje funkci na řetězec.

Vlastnost Function.prototype

Prototyp pro třídu objektů.

Funkce getClass()

Vrací třídu JavaClass objektu JavaScript.

Funkce isNaN()

Kontroluje, zda jde o ne-číslo (Not-a-Number).

h. Objekt Number

Pouzdru na numerické konstanty.

Number.MAX_VALUE

Největší číslo, vyjádřitelné v JavaScriptu (asi

1,79E+308).

Number.MIN_VALUE

Nejmenší číslo, které je vyjádřitelné v JavaScriptu (asi

2,22E-308).

Number.NaN

Zvláštní hodnota, udávající, že výsledek mat. operace je nečíslo.

Number.NEGATIVE_INFINITY Záporné nekonečno.

Number.POSITIVE_INFINITY Nekonečno.

Number.toString() Převéde číslo na řetězec.

Number.valueOf() Primitivní hodnota daného objektu.

i. Objekt Object

Charakteristika všech objektů JavaScriptu.

Konstruktor

new Object();

new Object(value);

Vlastnosti

constructor Odkaz na funkci, která byla konstruktorem daného objektu.

Metody

assign() Když je tato metoda definována, tak je použita k implementaci přiřazovacího operátoru (=) JavaScriptu.

eval() Vyhodnotí řetězec kódu JavaScriptu v kontextu daného objektu.

toString() Konverze objektu na řetězec.

valueOf() Vráť primitivní hodnotu objektu, pokud existuje. U objektů typu Object vrátí samotný objekt.

j. Funkce unescape()

Dekódování řetězce, zakódovaného pomocí escape().

k. Objekt URL

Univerzální zdrojový lokátor.

Přehled

window.location

document.links[i]

Vlastnosti

hash Řetězec-hešovací část URL, včetně hešovací značky (#). tato část určuje název kotvy v objektu, na nějž je odkazováno prostřednictvím URL.

host část URL – název hostitele a port

hostname název hostitele

href kompletní URL

pathname část URL – cesta

port port

protocol část URL – protocol, včetně dvojtečky

search vyhledávací nebo dotazovací část URL, včetně otazníku

Operátor typeof

Hodnota tohoto operátoru je řetězec, reprezentující datový typ operandu.

i. Možné hodnoty:

number

string

object

function

undefined

Přístupové operátory polí a objektů

K polím přistupujeme pomocí operátoru [].

K prvkům objektu můžeme přistoupit tak, že použijeme tečku (.). S oběma je nakládáno jako s operátory.

Operátor . očekává objekt jako svůj levý operand a název objektové vlastnosti nebo metody jako pravý operand. Tento pravý operand by neměl být řetězec nebo proměnná, která obsahuje řetězec.

Příklady

document.lastModified

navigator.appName

frames[0].length

document.write("ahoj světe")

Operátor [] umožňuje přístup k prvkům pole a k vlastnostem objektů

Příklad

document["lastModified"]

frames[0]['length']

data["val"+i]

Když je použito jako asociativní pole, nelze použít ., ale [].

I. Objekt history

Objekt history je pole URL v prohlížečové historii okna rámečku. Pro okno nevyšší úrovně je objekt history prezentací **Go** prohlížeče.

Vlastnosti

current – řetězec, udávající URL aktuálního dokumentu

length – počet URL uložených v objektu history

next – řetězec, udávající URL dokumentu následujícího v seznamu historie

previous – řetězec, udávající URL dokumentu předcházejícímu v seznamu historie

Metody

back() – krok zpět na předchozí navštívené URL

forward() – krok vpřed na dříve navštívené URL

go() – jdi na některé dříve navštívené URL

toString – vrátí tabulku ve formátu HTML, obsahující historii prohlížečového okna

Příklady použití

history.back – krok zpět

history.go(-2) – načte druhé předcházející URL z historie

history.go(2) – načte druhé následující URL z historie

history[0] – první URL zobrazené v aktuálním okně

history[history.length-1] – poslední URL zobrazené v aktuálním okně

history.current – aktuálně zobrazené URL

history.go – znovu načte aktuální URL

i. Objekty Navigator

Vlastnost **window.navigator** odkazuje na objekt Navigator, který obsahuje informace o webovém prohlížeči jako celku (verze, seznam datových formátů, které může zobrazit).

Vlastnosti

appName – název prohlížeče

appVersion – informace o čísle verze a platformě prohlížeče

userAgent – řetězec, který prohlížeč posílá ve svém záhlaví USER-AGENT v požadavcích http

appName – kódový název prohlížeče, který není zpravidla zvlášť užitečný

Další příkazy JavaScriptu

With

Nastaví objekt jako defaultní pro příkazy. Libovolný odkaz na vlastnost bez objektu je automaticky považován za odkaz na objekt.

Syntaxe

```
with (objekt){  
    příkazy  
}
```

m. Příklad použití objektu Math

```
with (Math) {  
    a = PI * r*r  
    x = r * cos(theta)  
    y = r * sin(theta)  
}
```

<HTML>

```

<HEAD>
<TITLE>Matematické funkce v praxi</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
  with (Math)
  {
    as=sin(PI)*tan(PI/2);
    od=sqrt(16);
    pk=PI*pow(0.5,2);
    nc=random()*100;
  }
  with (document)
  {
    write("sinus PI * tangens PI/2 = "+as);
    write("<BR>druhá odmocnina z čísla 16 = "+od);
    write("<BR>obsah kruhu s poloměrem 0.5 = "+pk);
    write("<BR>náhodné číslo od 0 do 100 = "+nc);
  }
// -->
</SCRIPT>
</BODY>
</HTML>

```

Výsledek je pak:

```

sinus PI * tangens PI/2 = 2
druhá odmocnina z čísla 16 = 4
obsah kruhu s poloměrem 0.5 = 0.7853981633974483
náhodné číslo od 0 do 100 = 30.528836477465028

```

```

<HTML>
<HEAD>
<TITLE>Výpočet faktoriálu</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function faktorial(n)
{
  if (n==0 || n==1) return 1;
  else return n*faktorial(n-1);
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
  cislo=window.prompt("Zadej číslo v rozsahu 0 až 170",""); window.alert(cislo+"!
  =" +faktorial(cislo));
// -->

```

```
</SCRIPT>
</BODY>
</HTML>
```

n. This

Klíčové slovo, které můžeme použít při odkazu na aktuální objekt. Obvykle v metodách odkazuje this na volající objekt.

Syntaxe

this[vlastnost]

o. Objekt history

Objekt history je pole URL v prohlížečové historii okna rámečku. Pro okno nevyšší úrovně je objekt history prezentací **Go** prohlížeče.

Vlastnosti

- current – řetězec, udávající URL aktuálního dokumentu
- length – počet URL uložených v objektu history
- next – řetězec, udávající URL dokumentu následujícího v seznamu historie
- previous – řetězec, udávající URL dokumentu předcházejícímu v seznamu historie

Metody

- back() – krok zpět na předchozí navštívené URL
- forward() – krok vpřed na dříve navštívené URL
- go() – jdi na některé dříve navštívené URL
- toString – vrátí tabulku ve formátu HTML, obsahující historii prohlížečového okna

Příklady použití

history.back – krok zpět
history.go(-2) – načte druhé předcházející URL z historie
history.go(2) – načte druhé následující URL z historie
history[0] – první URL zobrazené v aktuálním okně
history[history.length-1] – poslední URL zobrazené v aktuálním okně
history.current – aktuálně zobrazené URL
history.go – znovu načte aktuální URL

i. Příklad vpřed, zpět

```
<html>
<head>
<title>Programing Page - Script 03</title>
</head>
<body>
<CENTER>
```



```
<A HREF="javascript:history.back()">&lt;&lt;&lt; Zpět &lt;&lt;&lt;</A> <A  
HREF="javascript:history.forward()">&gt;&gt;&gt; Vpřed &lt;&lt;&lt;</A>  
</CENTER>  
</body>  
</html>
```

Stránka pak má tento vzhled. Po klepnutí na Zpět, případně Vpřed prohlížeč zobrazí předchozí, případně následující stránku ze své historie.

<<< Zpět <<< >>> Vpřed <<<

ii. Objekty Navigator

Vlastnost **window.navigator** odkazuje na objekt Navigator, který obsahuje informace o webovém prohlížeči jako celku (verze, seznam datových formátů, které může zobrazit).

Vlastnosti

- **appName** – název prohlížeče
- **appVersion** – informace o čísle verze a platformě prohlížeče
- **userAgent** – řetězec, který prohlížeč posílá ve svém záhlaví USER-AGENT v požadavcích http
- **appCodeName** – kódový název prohlížeče, který není zpravidla zvlášť užitečný

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
```

```
<HTML>
<HEAD>
<META HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=windows-1250">
<TITLE>Přesměrování na optimalizovanou stránku</TITLE>
<SCRIPT LANGUAGE="JavaScript" TYPE="text/javascript">
<!--
with (navigator) {
navez=appName; verze=parseInt(appVersion); }
with (window) {
if (navez=="Microsoft Internet Explorer" && verze>=4) location="ie4.htm";
    else if (navez=="Netscape" && verze>=4) location="nn4.htm";
    else location="ine.htm"; }
// -->
</SCRIPT>
</HEAD>
<BODY>
Optimalizovaná stránka pro prohlížeče nepodporující JavaScript
</BODY>
</HTML>
```

Výsledek pak je

Optimalizovaná stránka pro Netscape Navigator 4.0+

p. Další příkazy JavaScriptu

This

Klíčové slovo, které můžeme použít při odkazu na aktuální objekt. Obvykle v metodách odkazuje this na volající objekt.

Syntaxe

this[vlastnost]

q. Objekt Image

i. Vlastnosti objektu Image

border	Celé číslo pouze ke čtení, udává v bodech šířku okraje kolem obrázku. Tato hodnota se nastavuje atributem BORDER .
complete	Logická hodnota pouze ke čtení, udávající, zda je obrázek celý nahrán.
height	Celé číslo pouze ke čtení, udávající v bodech výšku obrázku. Tato hodnota se nastavuje atributem HEIGHT .
hspace	Celé číslo pouze ke čtení, udávající v bodech horizontálně množství místa navíc, přidaného k pravému a levému okraji obrázku. Tato hodnota se nastavuje atributem HSPACE .
lowsrc	Řetězec ke čtení a zápisu, udávající URC náhradního obrázku pro zobrazení s nízkým rozlišením.
name	Řetězec pouze ke čtení, definovaný HTML-atributem NAME , udává název obrázku. Pokud obrázek dostane prostřednictvím atributu NAME název, je odkaz na tento obrázek umístěn navíc kromě pozice v poli document.images[] také ve vlastnosti image-name .
src	Řetězec ke čtení a zápisu, udávající URL obrázku, který má být zobrazen. Jeho počáteční hodnota je nastavena atributem SRC .
vspace	Celé číslo pouze ke čtení, udávající bodech vertikálně množství místa navíc, přidaného nad a pod obrázek. Tato hodnota se nastavuje atributem VSPACE .
width	Celé číslo pouze ke čtení, udávající v bodech šířku obrázku. tato hodnota se nastavuje atributem WIDTH .

Události ovladače

onabort	Vyvolán, pokud uživatel přeruší načítání obrázku.
onerror	Vyvolán, pokud při načítání obrázku dojde k nějaké chybě.
onload	Vyvolán, pokud je nahrávání obrázku úspěšně ukončeno.

Syntaxe objektu Image v HTML

Objekt Image je vytvořen standardní HTML značkou , navíc s událostními ovladači. Některé atributy budou v následující syntaxi vynechány, protože nejsou JavaScriptem použity nebo mu nejsou přístupné.

<IMG SRC="url"	obrázek určený k zobrazení
WIDTH=pixels	šířka obrázku
HEIGHT=pixels	výška obrázku
[NAME="image_name"]	vlastnost název obrázku
[LOWSRC="url"]	náhradní obrázek s nízkým rozlišením
[BORDER=pixels]	šířka okraje obrázku
[HSPACE=pixels]	místo navíc kolem obrázku horizontálně
[VSPACE=pixels]	místo navíc kolem obrázku vertikálně
[onLoad=handler]	vyvolán po úplném natažení obrázku
[onError=handler]	vyvolán při chybě během natahování obrázku
[onAbort=handler]	vyvolán při přerušení natahování uživatelem
>	

ii. Praktické příklady

1. Změna obrázku

k11-p1.htm

<HTML>

<HEAD>

<TITLE>Záměna dvou obrázků</TITLE>

```

<SCRIPT LANGUAGE="JavaScript">
<!--
function on() {
    document.images[0].src="on.gif"; }
function off() {
    document.zarovka.src="off.gif"; }
// -->
</SCRIPT>
</HEAD>
<BODY>
<A HREF="" onMouseOver="on();" onMouseOut="off();">
<IMG SRC="off.gif" NAME="zarovka" BORDER="0"></A>
</BODY>
</HTML>

```

2. Cache s obrázky

k11-p1cache.htm

```

<HTML>
<HEAD>
<TITLE>Záměna dvou obrázků</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
obr1=new Image(); obr1.src="on.gif";
obr2=new Image(); obr2.src="off.gif";

function on()
{ document.images[0].src="on.gif";
}

function off()
{ document.zarovka.src="off.gif";
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<A HREF="" onMouseOver="on();" onMouseOut="off();">
<IMG SRC="off.gif" NAME="zarovka" BORDER="0"></A>
</BODY>
</HTML>

```